Keywords: vision-language-action • robot policy • real-time inference • efficient VLA

TurboVLA: Real-Time Vision-Language-Action Model at 32 Hz on an RTX 4090 with <1 GB VRAM

Eliminating the LLM Bottleneck in Robot Policies via Direct V+L→A Mapping Yields 97.7% on LIBERO with Only 0.2B Parameters

By Hengyi Xie, Chenfei Yao, Xianjin Wu, Xuanyang Xi, Yiping Tang, Di Xu, Yingying Zhu, Dingkang Liang, Xiang Bai, Han Ding (Huazhong Univ. of Science and Technology; Huawei Technologies)

arXiv:2607.27205 • July 30, 2026

State-of-the-art robot manipulation models route all perception through a billion-parameter language model—incurring hundreds of milliseconds of latency and gigabytes of VRAM at every policy step. **TurboVLA** breaks from this convention, replacing the V→L→A pathway with a direct V+L→A mapping using a compact bidirectional interaction module. The result runs at 32 Hz on a consumer RTX 4090 with <1 GB VRAM while achieving 97.7% average success on LIBERO—matching models 35× larger.

AT A GLANCE

Problem: LLM-centric VLA policies incur prohibitive compute and memory overhead at every inference step, blocking real-time deployment.

Why it matters: Real-time control at 32 Hz on consumer hardware is a prerequisite for affordable robot deployment outside research labs.

Method: Independent visual and language encoders fused via lightweight bidirectional cross-attention; continuous action chunks predicted by a compact decoder.

Result: 97.7% on LIBERO, 31.2 ms per step, 0.9 GB VRAM, 0.2B parameters.

Evidence: Matches or exceeds OpenVLA-7B and Pi-0 (3.3B) while running 7–35× faster and consuming 7–16× less memory.

The Big Picture

Vision-language-action models represent the current frontier of generalist robot policies: by building on pre-trained vision-language models, they inherit rich semantic representations that generalize across diverse manipulation tasks. Papers like OpenVLA and Pi-0 have demonstrated that leveraging multi-billion-parameter LLM backbones as the core of a robot policy can yield impressive task success rates.

The bottleneck, however, is clear: inference in these models runs at 2–10 Hz on research-grade GPU hardware, requiring tens of gigabytes of VRAM. Consumer robots, surgical systems, and industrial manipulators demand 30–100 Hz control with modest compute budgets. TurboVLA challenges the architectural assumption that the LLM must be in the critical path of every policy inference.

Why Existing Approaches Fall Short

The dominant V→L→A paradigm processes visual observations by projecting them into the LLM's token space: visual patches become "visual tokens" that attend to language tokens in the LLM's self-attention layers. This design has three costs:

Quadratic attention complexity. The joint visual-language sequence has $N_v + N_l$ tokens; self-attention costs $O((N_v + N_l)^2)$ per layer. For $N_v = 256$ visual tokens and $N_l = 30$ language tokens, this is $\sim 80{,}000$ token pairs per layer, repeated across all 32+ LLM layers.

Memory dominated by LLM weights. Even a 7B-parameter LLM at 4-bit quantization occupies ~ 3.5 GB—more than most embedded compute platforms support.

No separation of concerns. The LLM processes both semantic instruction understanding and low-level observation encoding in the same computational graph, making it impossible to specialize or prune either component independently.

Core Mechanism

TurboVLA introduces three specialized modules that replace the monolithic LLM backbone:

1. **Visual encoder (ViT-S/16):** Encodes the observation image into $N_v = 196$ patch tokens $V \in \mathbb{R}^{N_v \times d_v}$.
2. **Language encoder (CLIP text, frozen):** Encodes the instruction into $N_l \approx 30$ tokens $L \in \mathbb{R}^{N_l \times d_l}$.
3. **Bidirectional Vision-Language Interaction (BiVLI):** Applies two rounds of alternating cross-attention, letting V attend to L and L attend to V, producing aligned representations V' and L'.

The BiVLI module contains only ∼50M parameters. The action decoder takes $[\text{pool}(V') \oplus \text{pool}(L')]$ and predicts a chunk of $H = 10$ continuous actions.

Technical Formulation

For round $r \in \{1, 2\}$ of BiVLI:

$$V^{(r)} = V^{(r-1)} + \text{MHA}_{V \leftarrow L}(Q{=}V^{(r-1)},\ K{=}L^{(r-1)},\ V{=}L^{(r-1)})$$

$$L^{(r)} = L^{(r-1)} + \text{MHA}_{L \leftarrow V}(Q{=}L^{(r-1)},\ K{=}V^{(r)},\ V{=}V^{(r)})$$

where MHA is multi-head attention. The interaction cost is $O(N_v \cdot N_l)$ per round—30× lower than full self-attention over the combined sequence.

The combined representation $z = \text{pool}(V^{(2)}) \oplus \text{pool}(L^{(2)})$ is decoded to an action chunk:

$$\hat{a}_{1:H} = \text{Decoder}(z), \qquad \hat{a}_{1:H} \in \mathbb{R}^{H \times d_a}$$

Training minimizes the behavior cloning loss:

$$\mathcal{L}_{\text{BC}} = \sum_{h=1}^{H} \left\| \hat{a}_h - a_h^* \right\|_2^2$$

over expert demonstrations.

Learning or Inference Procedure

Training uses two stages. In the first (pretraining) stage, the visual encoder is initialized from CLIP ViT-S/16 and the BiVLI module is randomly initialized; both are trained on the OpenX-Embodiment dataset (∼1M demonstrations). In the second (fine-tuning) stage, the full model including the visual encoder is fine-tuned on task-specific demonstrations in the target environment (LIBERO, ∼500 demonstrations per suite). The CLIP text encoder is frozen throughout.

At inference, the visual encoder, BiVLI, and action decoder run in a single forward pass per time step—no autoregressive decoding, no KV cache management, no LLM overhead.

What the Guarantee Says

By design, the backbone parameters p_θ are never called at inference time. The only per-step compute is ViT-S forward (12 ms), BiVLI two-round cross-attention (11 ms), and MLP action decoder (8 ms), totaling 31.2 ms—safely below the 33 ms deadline for 30 Hz control. This is a structural bound, not a statistical approximation: no matter how complex the manipulation task, the per-step latency is fixed by architecture rather than by task difficulty.

Experimental Findings

LIBERO benchmark (5 suites, higher is better):

Model	Params	Avg.	ms/step	VRAM
OpenVLA-7B	7B	96.1%	218	14.0 GB
Pi-0 (distil.)	3.3B	97.2%	82	6.4 GB
TurboVLA	**0.2B**	**97.7%**	**31**	**0.9 GB**

TurboVLA achieves the best average success rate while using 35× fewer parameters, running 7× faster, and consuming 7–16× less memory.

Cross-embodiment: Evaluated on Franka, xArm, and UR5, TurboVLA shows less catastrophic forgetting between embodiments than OpenVLA, attributed to the frozen CLIP text encoder providing a stable semantic anchor during multi-embodiment fine-tuning.

Ablations and Interpretation

BiVLI rounds: $R = 1$ round drops average success by 3.2%; $R = 4$ rounds adds only 0.4% while doubling BiVLI latency. Two rounds is the Pareto-optimal choice for accuracy/latency trade-off.

Visual encoder size: ViT-B/16 (3× more parameters) improves success by 1.1% but doubles VRAM. ViT-S/16 is appropriate for resource-constrained deployment.

Action chunk horizon: $H = 10$ gives the best task completion; shorter horizons ($H = 4$) hurt due to high-frequency control jitter and longer horizons ($H = 20$) hurt due to compounding prediction error.

Reference

Hengyi Xie, Chenfei Yao, Xianjin Wu, Xuanyang Xi, Yiping Tang, Di Xu, Yingying Zhu, Dingkang Liang, Xiang Bai, Han Ding. **TurboVLA: Real-Time Vision-Language-Action Model at 32 Hz on an RTX 4090 with <1 GB VRAM**. arXiv:2607.27205, July 30, 2026. https://arxiv.org/abs/2607.27205